LAPORAN PENGEMBANGAN MICROSERVICES UNTUK USER MANAGEMENT SERVICES

Untuk Memenuhi Salah Satu Tugas Lab. Pemrograman Web Lanjut

Ditulis Oleh:

Nama: Helena Jemima Widjaja

NPM: 232310042

Kelas: TI - 23 - KA

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS INFORMATIKA DAN PARIWISATA

INSTITUT BISNIS DAN INFORMATIKA KESATUAN

2026

1. PENDAHULUAN

Pada era perkembangan teknologi saat ini, sistem aplikasi banyak dikembangkan menggunakan arsitektur microservices untuk meningkatkan fleksibilitas, skalabilitas, dan kemudahan dalam pengelolaan sistem. Salah satu komponen penting dalam sistem tersebut adalah layanan pengelolaan data pengguna atau *User Management Service*. Layanan ini bertanggung jawab untuk mengatur berbagai informasi pengguna yang digunakan dalam proses autentikasi, manajemen profil, serta pengelolaan akun.

Pada tugas ini, dikembangkan sebuah User Management Service menggunakan Node.js dan Express.js dengan menerapkan konsep RESTful API. Selain itu, digunakan database NoSQL yaitu MongoDB untuk menyimpan data pengguna. Penggunaan MongoDB dipilih karena memiliki struktur data yang fleksibel dan mampu menangani data yang bersifat dinamis, sehingga cocok untuk kebutuhan aplikasi modern berbasis microservices.

Melalui pengembangan layanan ini, diharapkan mahasiswa dapat memahami bagaimana membangun backend service sederhana yang mampu melakukan pengelolaan data pengguna secara efektif dan efisien.

2. TUJUAN

Adapun tujuan dari pembuatan User Management Service ini adalah sebagai berikut:

1. Memahami konsep dasar arsitektur microservices dalam pengembangan aplikasi.
2. Mengimplementasikan RESTful API menggunakan Node.js dan Express.js.
3. Menggunakan database NoSQL (MongoDB) untuk penyimpanan data pengguna.
4. Mengembangkan fitur CRUD (Create, Read, Update, Delete) pada data pengguna.
5. Melakukan pengujian API menggunakan tools seperti Postman.

3. TEKNOLOGI YANG DIGUNAKAN

Dalam pengembangan *User Management Service* ini, digunakan beberapa teknologi utama yang mendukung proses pembuatan dan pengujian sistem, yaitu sebagai berikut:

1. Node.js

   Node.js digunakan sebagai platform backend untuk menjalankan JavaScript di sisi server. Teknologi ini memungkinkan pembuatan aplikasi yang cepat dan efisien serta mendukung arsitektur non-blocking yang cocok untuk pengembangan API.
2. Express.js

   Express.js merupakan framework dari Node.js yang digunakan untuk mempermudah pembuatan RESTful API. Dengan Express, proses routing, middleware, dan pengelolaan request-response menjadi lebih sederhana dan terstruktur.
3. MongoDB

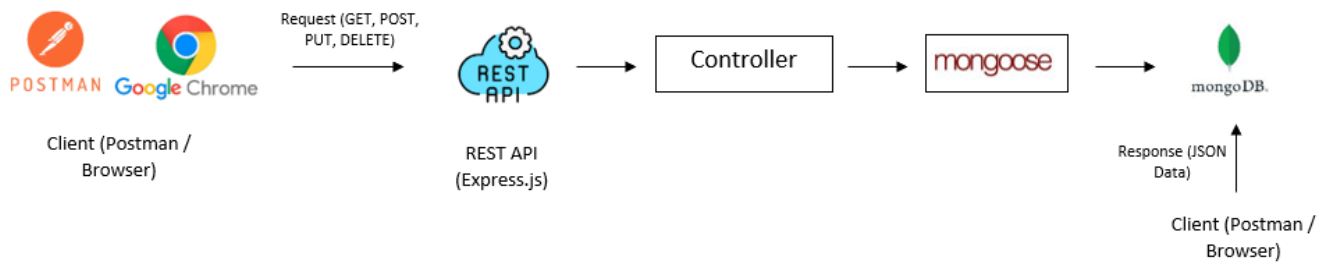
   MongoDB adalah database NoSQL yang digunakan untuk menyimpan data pengguna. Database ini memiliki struktur berbasis dokumen (JSON) yang fleksibel sehingga memudahkan dalam pengelolaan data yang bersifat dinamis.
4. Mongoose

   Mongoose digunakan sebagai Object Data Modeling (ODM) untuk MongoDB. Library ini membantu dalam mendefinisikan schema, validasi data, serta mempermudah interaksi antara aplikasi Node.js dengan database MongoDB.
5. Postman

   Postman digunakan sebagai tools untuk melakukan pengujian API. Dengan Postman, setiap endpoint seperti GET, POST, PUT, dan DELETE dapat diuji untuk memastikan sistem berjalan dengan baik.

4. ARSITEKTUR SISTEM

Arsitektur sistem pada *User Management Service* ini menggunakan pendekatan microservices sederhana dengan komunikasi berbasis RESTful API. Sistem ini terdiri dari beberapa komponen utama yaitu client, server (backend), dan database. Secara umum, alur kerja sistem dapat dijelaskan sebagai berikut:



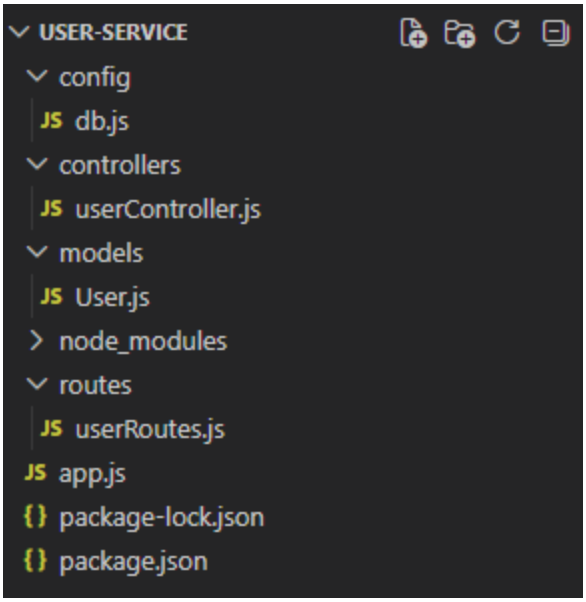
Client (seperti browser atau Postman) mengirimkan request ke server melalui endpoint API yang telah disediakan. Request tersebut kemudian diterima oleh Express.js dan diteruskan ke bagian controller untuk diproses sesuai dengan fungsi yang diminta, seperti mengambil data atau menambahkan user baru. Selanjutnya, controller akan berinteraksi dengan database MongoDB melalui Mongoose untuk melakukan operasi CRUD. Setelah proses selesai, server akan mengirimkan response kembali ke client dalam format JSON.

Arsitektur ini memisahkan antara logika aplikasi, pengelolaan data, dan komunikasi API sehingga sistem menjadi lebih terstruktur dan mudah untuk dikembangkan. Selain itu, penggunaan MongoDB sebagai database NoSQL memungkinkan fleksibilitas dalam penyimpanan data pengguna yang dapat berubah-ubah sesuai kebutuhan.

5. STRUKTUR PROJECT

Struktur project pada *User Management Service* disusun secara modular untuk memisahkan setiap bagian berdasarkan fungsinya. Hal ini bertujuan agar kode lebih terorganisir, mudah dipahami, dan mudah dikembangkan di masa mendatang.

Berikut adalah struktur folder dari project yang dibuat:



1. Models: Folder ini digunakan untuk menyimpan model atau schema database. Pada project ini terdapat file User.js yang berisi struktur data user seperti username, email, dan password.
2. Controllers: Folder ini berisi logika utama dari aplikasi. Setiap fungsi seperti menambah, mengambil, mengupdate, dan menghapus data user ditulis di dalam controller.
3. Routes: Folder ini digunakan untuk mendefinisikan endpoint API. Didalamnya terdapat file userRoutes.js yang mengatur routing seperti GET, POST, PUT, dan DELETE.
4. Config: Folder ini berisi konfigurasi aplikasi, khususnya koneksi ke database MongoDB yang terdapat pada file db.js.
5. App.js : File utama yang digunakan untuk menjalankan server. Di dalam file ini dilakukan konfigurasi Express, pemanggilan database, serta penggabungan semua routes.
6. Package.json: File ini berisi informasi project serta daftar dependencies (library) yang digunakan seperti Express, Mongoose, dan lainnya.

6. IMPLEMENTASI

Pada tahap implementasi, dilakukan pembuatan User Management Service menggunakan Node.js, Express.js, dan MongoDB. Sistem ini mengimplementasikan konsep RESTful API untuk melakukan operasi CRUD (Create, Read, Update, Delete) terhadap data pengguna.

6.1. Koneksi Database

Koneksi ke database MongoDB dilakukan menggunakan Mongoose. File konfigurasi database berfungsi untuk menghubungkan aplikasi dengan database sehingga data dapat disimpan dan diambil.

Aplikasi akan mencoba terhubung ke MongoDB menggunakan URL lokal. Jika berhasil, akan muncul pesan “MongoDB Connected”.

6.2. Model User

Model digunakan untuk mendefinisikan struktur data yang akan disimpan di database. Struktur data user meliputi:

- username → nama pengguna
- email → email pengguna
- password → kata sandi

Model ini dibuat menggunakan Mongoose Schema yang memungkinkan validasi data serta pengaturan tipe data secara fleksibel.

6.3. Controller

Controller berisi logika utama dalam pengolahan data user. Pada bagian ini dibuat beberapa fungsi utama, yaitu:

- Create User → menambahkan data user baru ke database
- Get All Users → mengambil seluruh data user
- Get User by ID → mengambil data berdasarkan ID
- Update User → memperbarui data user
- Delete User → menghapus data user

Controller menerima request dari client melalui routes, kemudian memprosesnya dan berinteraksi dengan database menggunakan Mongoose.

6.4. Routes

Routes digunakan untuk mengatur endpoint API yang dapat diakses oleh client. Endpoint yang tersedia:

- GET /users → mengambil semua data user
- GET /users/:id → mengambil data berdasarkan ID
- POST /users → menambahkan user baru
- PUT /users/:id → mengupdate user

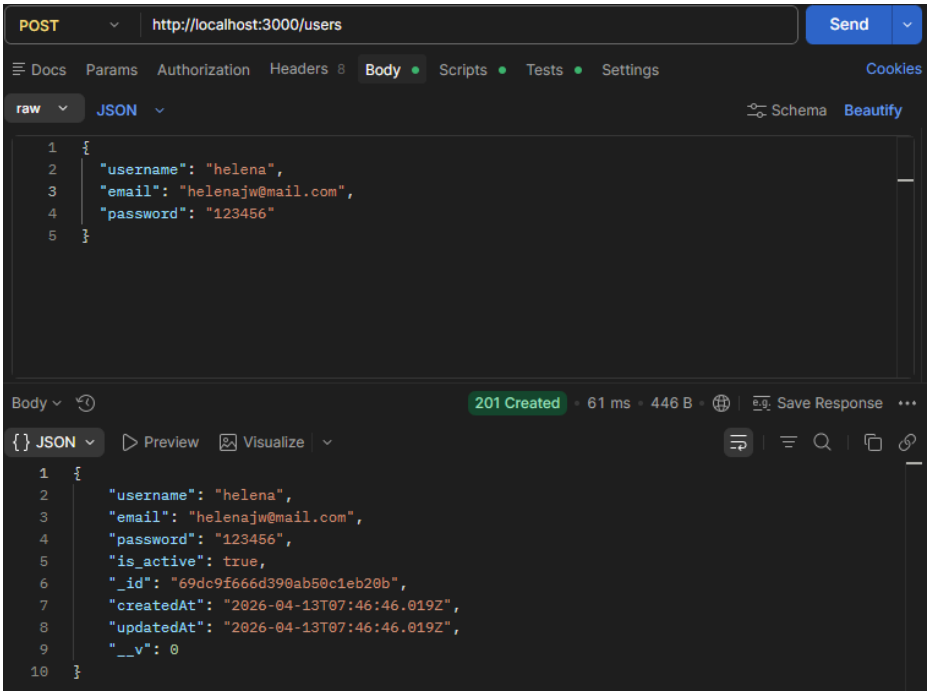
- DELETE /users/:id → menghapus user

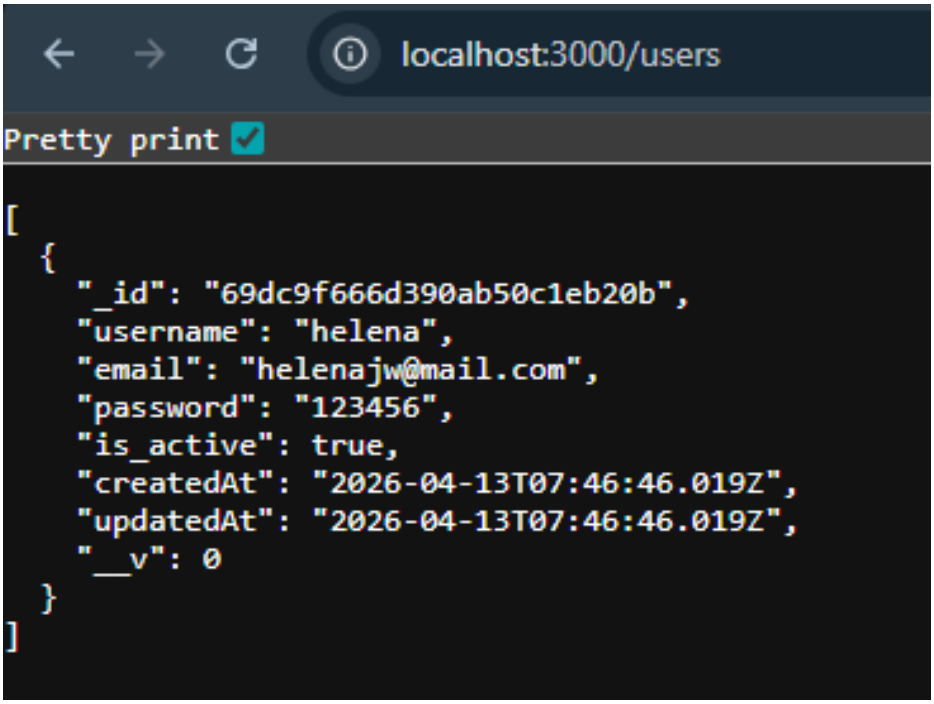
Routes menghubungkan URL endpoint dengan fungsi yang ada di controller sehingga setiap request dapat diproses dengan benar.

6.5. Pengujian API

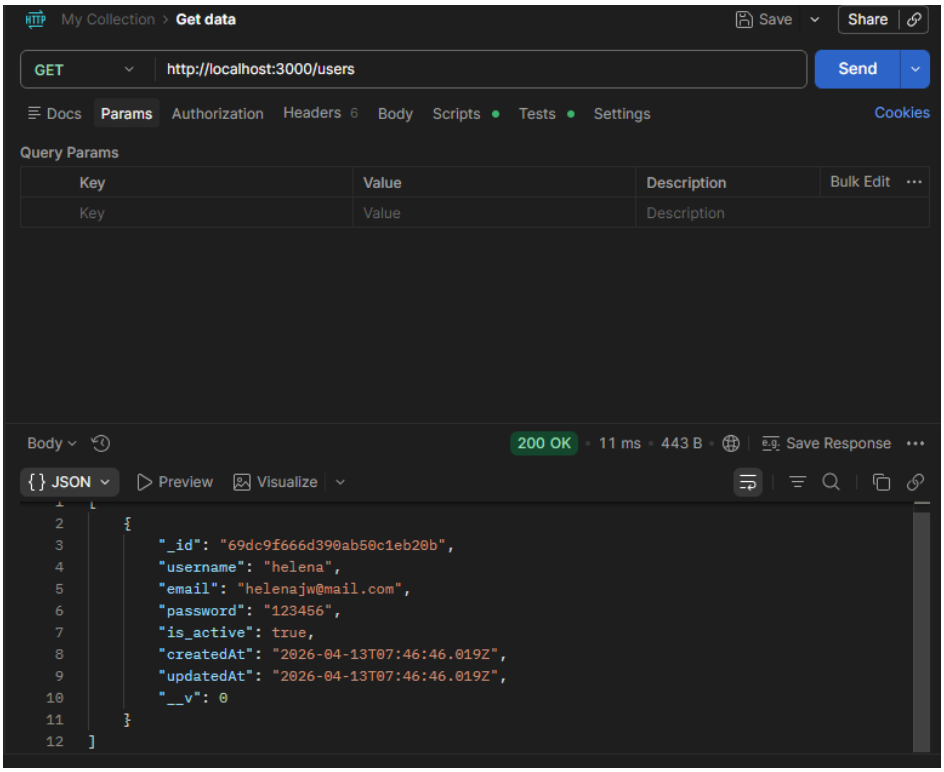
Pengujian dilakukan menggunakan Postman dengan mencoba seluruh endpoint yang telah dibuat. Langkah pengujian:

- Mengirim request POST untuk menambahkan user

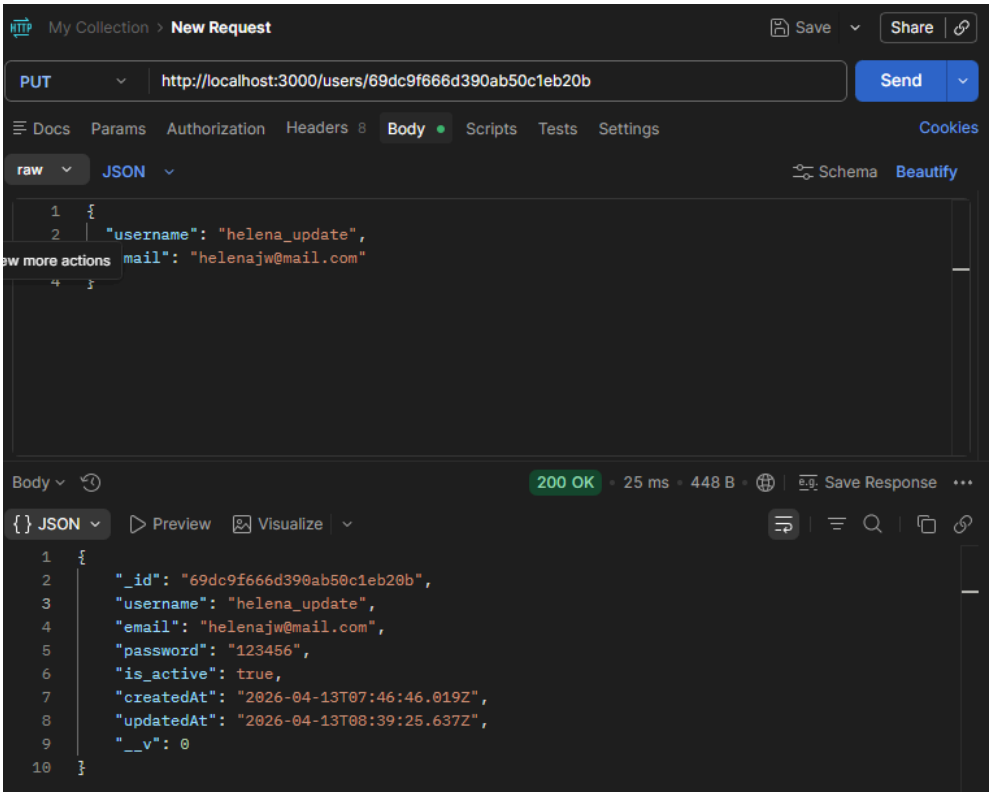


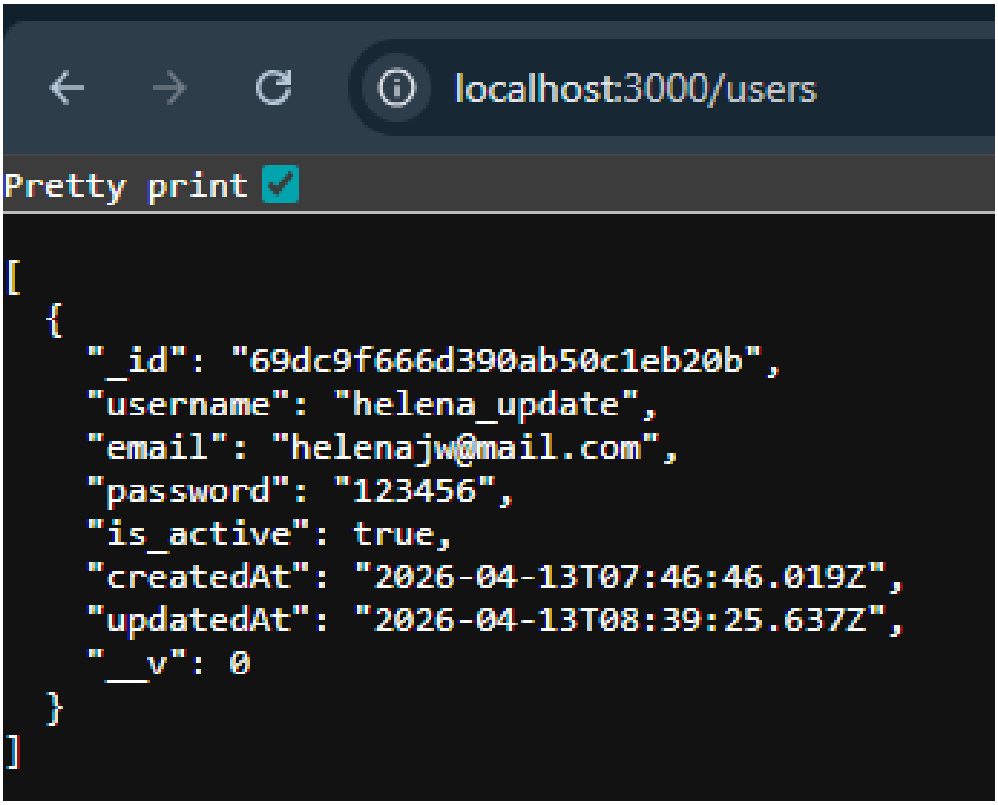


- Mengirim request GET untuk melihat data user

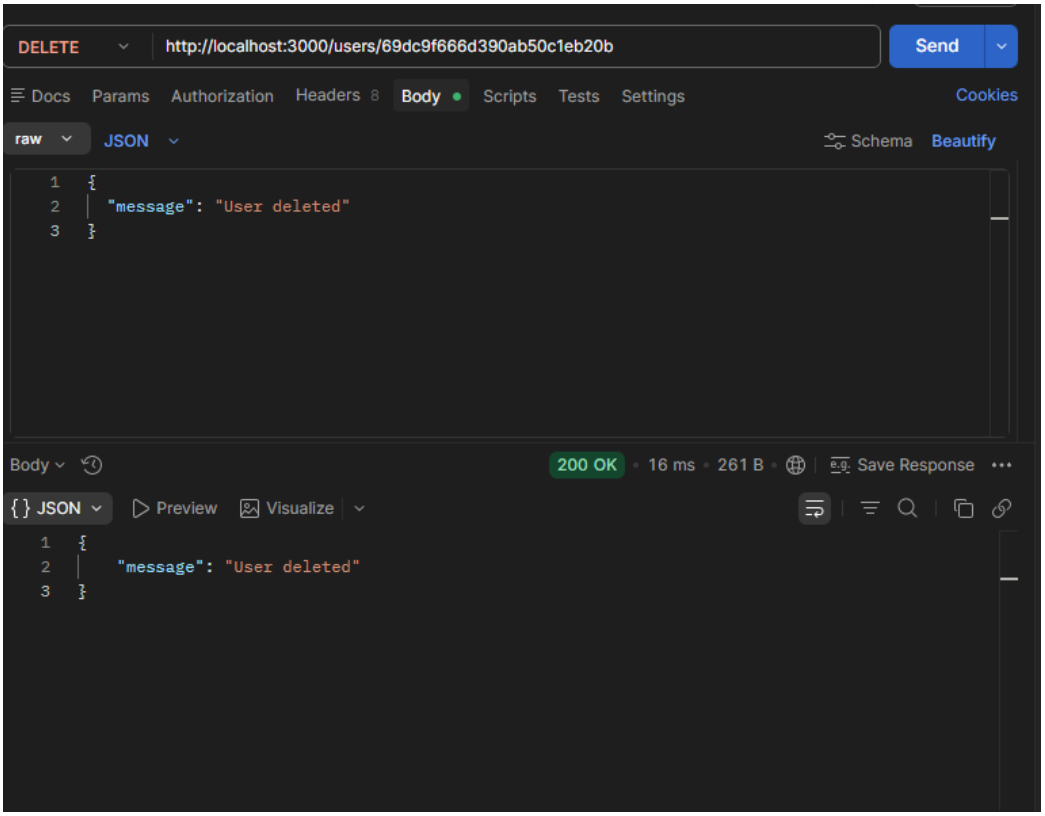


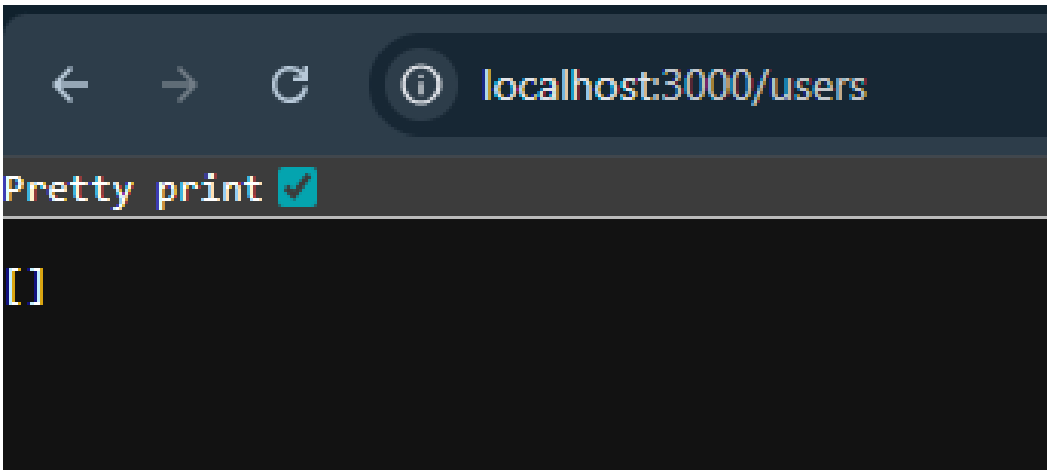
- Mengirim request PUT untuk mengupdate data





- Mengirim request DELETE untuk menghapus data





Semua endpoint berhasil dijalankan dan memberikan response sesuai dengan yang diharapkan dalam format JSON.

6.6. Alur Kerja Sistem

Secara keseluruhan, alur kerja sistem adalah sebagai berikut: Client mengirim request ke server melalui endpoint API, kemudian request diproses oleh routes dan controller. Selanjutnya, controller berinteraksi dengan database MongoDB untuk

melakukan operasi data. Setelah itu, hasil dari proses tersebut dikirim kembali ke client dalam bentuk response JSON.

7. HASIL DAN PEMBAHASAN

Pada tahap pengujian, *User Management Service* yang telah dikembangkan berhasil dijalankan dengan baik. Pengujian dilakukan menggunakan Postman untuk memastikan setiap endpoint API dapat berfungsi sesuai dengan yang diharapkan.

Hasil pengujian menunjukkan bahwa seluruh operasi CRUD dapat berjalan dengan benar. Pada saat pertama kali dilakukan pengujian endpoint **GET /users**, sistem menampilkan data kosong ([]), yang menandakan bahwa koneksi ke database telah berhasil namun belum terdapat data yang tersimpan. Setelah dilakukan penambahan data menggunakan endpoint **POST /users**, data pengguna berhasil disimpan ke dalam database MongoDB. Selanjutnya, ketika dilakukan pengujian kembali pada endpoint **GET /users**, data yang telah ditambahkan dapat ditampilkan dengan baik dalam format JSON.

Selain itu, pengujian pada endpoint **PUT /users/:id** menunjukkan bahwa data pengguna dapat diperbarui sesuai dengan input yang diberikan. Endpoint **DELETE /users/:id** juga berhasil menghapus data pengguna dari database. Hal ini membuktikan bahwa seluruh fungsi CRUD telah berjalan dengan baik dan sesuai dengan konsep RESTful API.

Secara keseluruhan, sistem yang dibangun telah mampu mengelola data pengguna dengan baik, mulai dari proses penambahan, pengambilan, pembaruan, hingga penghapusan data. Penggunaan MongoDB sebagai database NoSQL memberikan fleksibilitas dalam penyimpanan data, sementara penggunaan Express.js mempermudah dalam pembuatan API yang terstruktur.

8. KESIMPULAN

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa *User Management Service* berhasil dibangun menggunakan Node.js, Express.js, dan

MongoDB. Sistem ini mampu menjalankan seluruh operasi CRUD (Create, Read, Update, Delete) terhadap data pengguna dengan baik melalui RESTful API.

Penggunaan MongoDB sebagai database NoSQL memberikan fleksibilitas dalam penyimpanan data yang bersifat dinamis, sehingga sesuai untuk pengembangan aplikasi berbasis microservices. Selain itu, Express.js mempermudah dalam pengelolaan routing dan pembuatan endpoint API secara terstruktur.

Secara keseluruhan, sistem yang dikembangkan telah berjalan sesuai dengan tujuan yang diharapkan dan mampu memberikan pemahaman dasar mengenai implementasi backend service dalam arsitektur microservices.

9. LAMPIRAN

9.1. Coding

9.1.1. config/db.js

```
JS db.js ×
config > JS db.js > ...
1  const mongoose = require("mongoose");
2
3  const connectDB = async () => {
4    try {
5      await mongoose.connect("mongodb://127.0.0.1:27017/user_service");
6      console.log("MongoDB Connected");
7    } catch (err) {
8      console.error("DB ERROR:", err.message);
9      process.exit(1);
10   }
11 };
12
13 module.exports = connectDB;
```

9.1.2. controllers/userController.js

```
JS userController.js ●
controllers > JS userController.js > ...
1   const User = require("../models/User");
2
3   exports.createUser = async (req, res) => {
4     try {
5       const user = new User(req.body);
6       const saved = await user.save();
7       res.status(201).json(saved);
8     } catch (err) {
9       res.status(500).json({ message: err.message });
10    }
11  };
12
13  exports.getAllUsers = async (req, res) => {
14    const users = await User.find();
15    res.json(users);
16  };
17
18  exports.getUserById = async (req, res) => {
19    const user = await User.findById(req.params.id);
20    res.json(user);
21  };
22
23  exports.updateUser = async (req, res) => {
24    const user = await User.findByIdAndUpdate(req.params.id, req.body, { new: true });
25    res.json(user);
26  };
27
28  exports.deleteUser = async (req, res) => {
29    await User.findByIdAndDelete(req.params.id);
30    res.json({ message: "User deleted" });
31  };
```

9.1.3. models/User.js

```
JS User.js ×
models > JS User.js > ...
1   const mongoose = require("mongoose");
2
3   const userSchema = new mongoose.Schema({
4     username: { type: String, required: true },
5     email: { type: String, required: true },
6     password: { type: String, required: true },
7     is_active: { type: Boolean, default: true }
8   }, { timestamps: true });
9
10  module.exports = mongoose.model("User", userSchema);
```

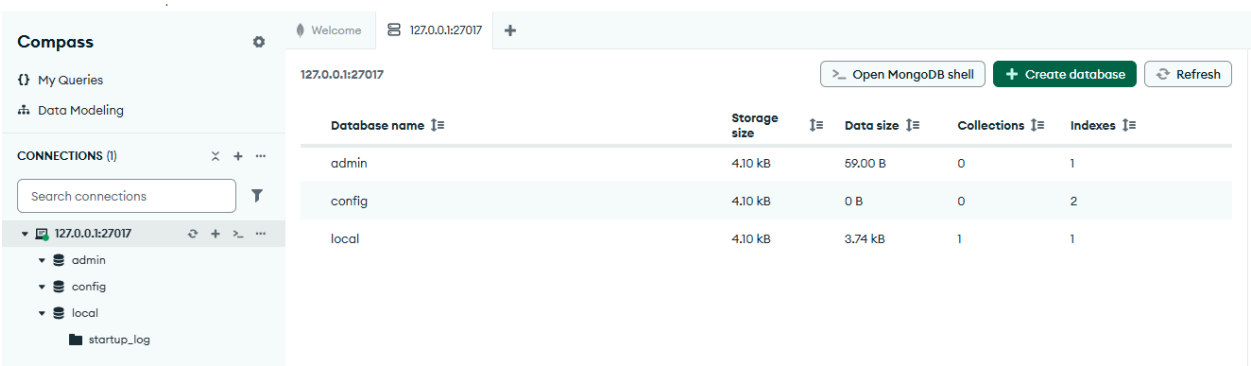
9.1.4. routes/userRoutes.js

```
JS userRoutes.js ×
routes > JS userRoutes.js > ...
1   const express = require("express");
2   const router = express.Router();
3   const controller = require("../controllers/userController");
4
5   router.get("/", controller.getAllUsers);
6   router.get("/:id", controller.getUserById);
7   router.post("/", controller.createUser);
8   router.put("/:id", controller.updateUser);
9   router.delete("/:id", controller.deleteUser);
10
11  module.exports = router;
```

9.1.5. app.js

```
JS app.js ×
JS app.js > ...
1   const express = require("express");
2   const bodyParser = require("body-parser");
3   const connectDB = require("./config/db");
4
5   const app = express();
6   connectDB();
7
8   app.use(bodyParser.json());
9
10  app.use("/users", require("./routes/userRoutes"));
11
12  app.listen(3000, () => {
13    console.log("Server running on port 3000");
14  });
```

9.2. Database


Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
127.0.0.1:27017
admin
config
local
startup_log
Welcome
127.0.0.1:27017
127.0.0.1:27017
Open MongoDB shell
Create database
Refresh
Database name
Storage size
Data size
Collections
Indexes
admin 4.10 kB 59.00 B 0 1
config 4.10 kB 0 B 0 2
local 4.10 kB 3.74 kB 1 1